

Lab02_222793

Rafael Bertagna da Silva

```
library(readr)
```

Warning: pacote 'readr' foi compilado no R versão 4.5.3

```
library(dplyr)
```

Warning: pacote 'dplyr' foi compilado no R versão 4.5.3

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

`filter`, `lag`

Os seguintes objetos são mascarados por 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
dir.create("dados", showWarnings = FALSE)
```

```
unzip(  
  "flights.csv.zip",  
  exdir = "dados"  
)
```

1 - Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

R: Para calcular o percentual de voos com `ARRIVAL_DELAY > 10`, para cada dia e companhia, são necessárias apenas as estatísticas do número total de voos válidos e o número de voos atrasados (`ARRIVAL_DELAY > 10`).

2 - Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`:

- a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
 - b. Remova observações que tenham valores faltantes em campos de interesse;
 - c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
 - d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`;
 - e. A função deve receber apenas dois argumentos:
 - f. `input`: o conjunto de dados (referente ao lote em questão);
- ii. `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
getStats <- function(input, pos) {  
  
  input %>%  
    filter(AIRLINE %in% c("AA", "DL", "UA", "US")) %>%  
    filter(!is.na(ARRIVAL_DELAY)) %>%  
    group_by(DAY, MONTH, AIRLINE) %>%  
    summarise(  
      n = n(),  
      atrasos = sum(ARRIVAL_DELAY > 10),  
      .groups = "drop"  
    )  
}
```

3 - Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

- a. Configure o tamanho do lote (`chunk`) para 100 mil registros;
- b. Configure a função de callback para instanciar `DataFrames` aplicando a função `getStats` criada acima;
- c. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```

resultado <- readr::read_csv_chunked(
  "dados/flights.csv",
  callback = readr::DataFrameCallback$new(getStats),
  chunk_size = 100000,
  col_types = readr::cols_only(
    AIRLINE = readr::col_character(),
    DAY = readr::col_integer(),
    MONTH = readr::col_integer(),
    ARRIVAL_DELAY = readr::col_double()
  )
)

```

4 - Crie uma função chamada computeStats que:

- a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
- b. Retorne as informações em um tibble contendo apenas as seguintes colunas:
- c. Cia: sigla da companhia aérea;
- ii. Data: data, no formato AAAA-MM-DD (dica: utilize o comando as.Date);
- iii. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo [0,1]

```

computeStats <- function(x) {

  x %>%
    group_by(DAY, MONTH, AIRLINE) %>%
    summarise(
      n = sum(n),
      atrasos = sum(atrasos),
      .groups = "drop"
    ) %>%
    mutate(
      Cia = AIRLINE,
      Data = as.Date(
        paste(2015, MONTH, DAY, sep = "-")
      ),
      Perc = atrasos / n
    ) %>%
    select(Cia, Data, Perc)
}

```

```
resultado_final <- computeStats(
  bind_rows(resultado)
)
```

5 - Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

- a. Instale e carregue os pacotes ggcal e ggplot2.
- b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
- c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
- d. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
 - ii. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
 - iii. Retornar para o usuário a base do calendário criada acima.
- iv. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.
- e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Título")`.

```
library(ggplot2)
```

Warning: pacote 'ggplot2' foi compilado no R versão 4.5.3

```
library(ggcal)

pal <- scale_fill_gradient(
  low = "#4575b4",
  high = "#d73027"
)

baseCalendario <- function(stats, cia) {

  dados_cia <- stats %>%
    filter(Cia == cia)
```

```

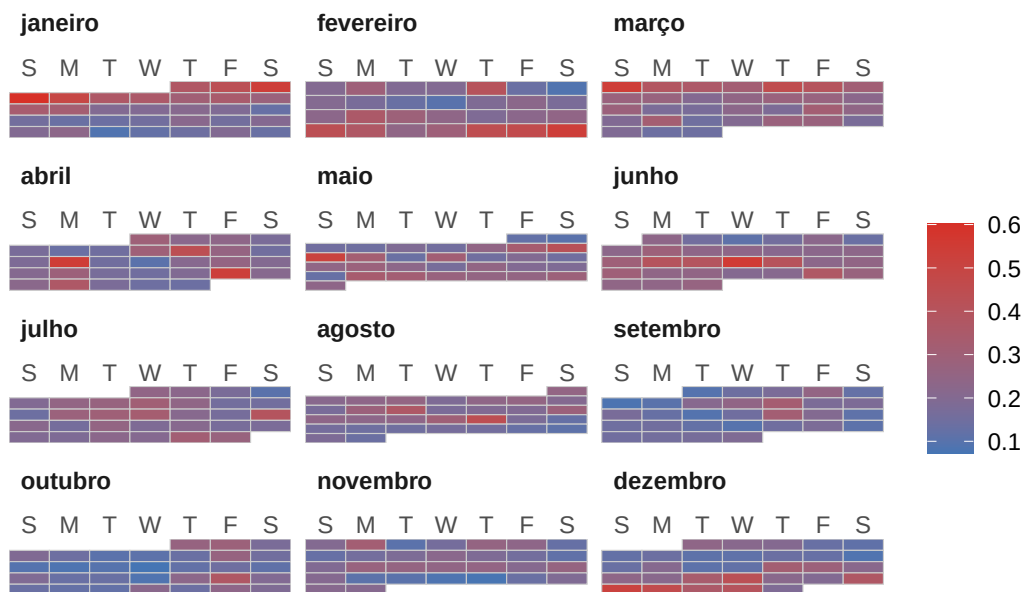
ggcal(
  dados_cia$Data,
  dados_cia$Perc
)
}

cAA <- baseCalendario(resultado_final, "AA")
cDL <- baseCalendario(resultado_final, "DL")
cUA <- baseCalendario(resultado_final, "UA")
cUS <- baseCalendario(resultado_final, "US")

cAA + pal + ggtitle("Percentual de voos atrasados - AA")

```

Percentual de voos atrasados – AA

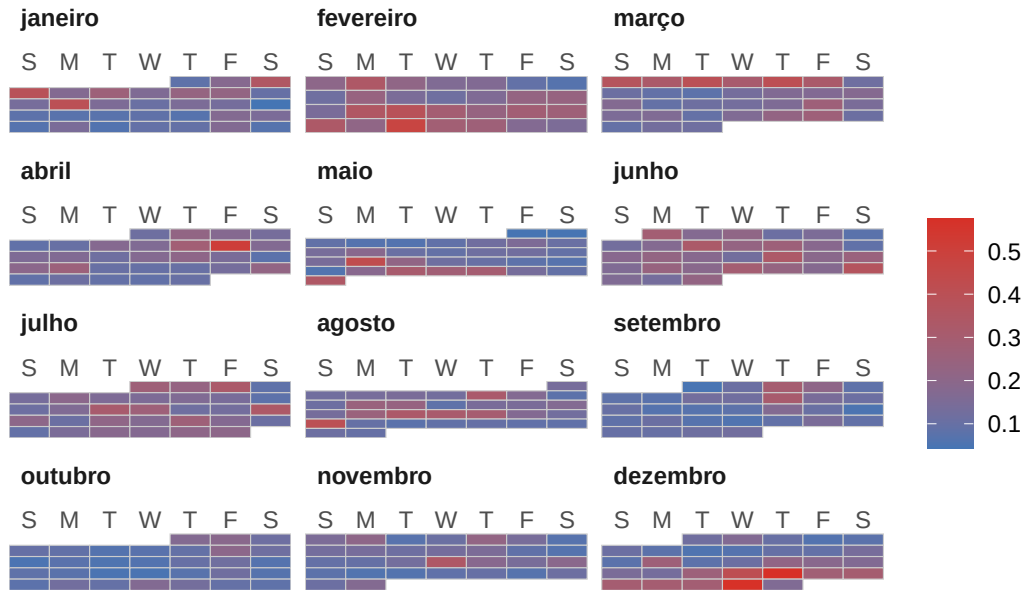


```

cDL + pal + ggtitle("Percentual de voos atrasados - DL")

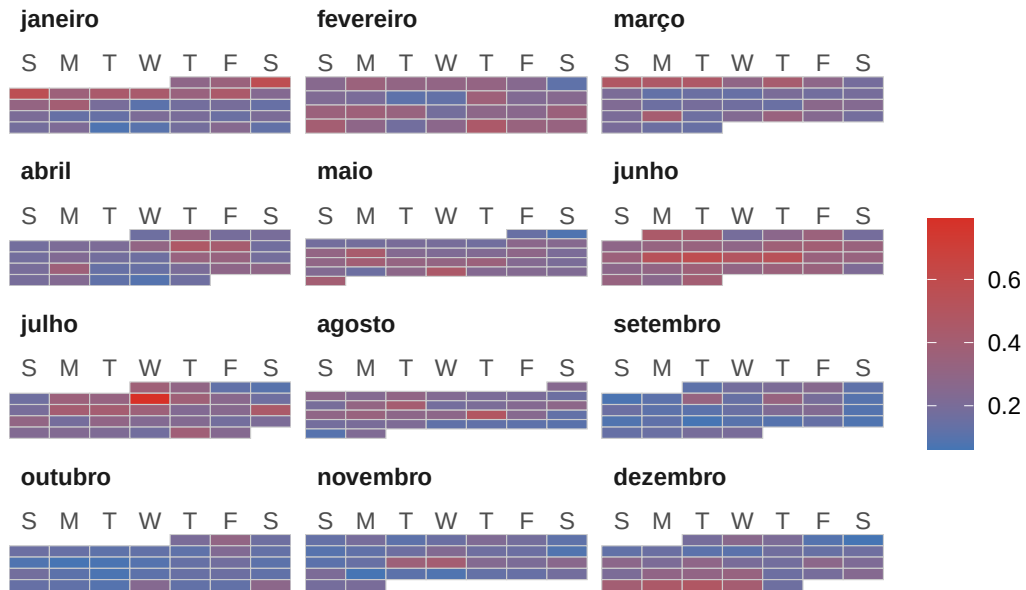
```

Percentual de voos atrasados – DL



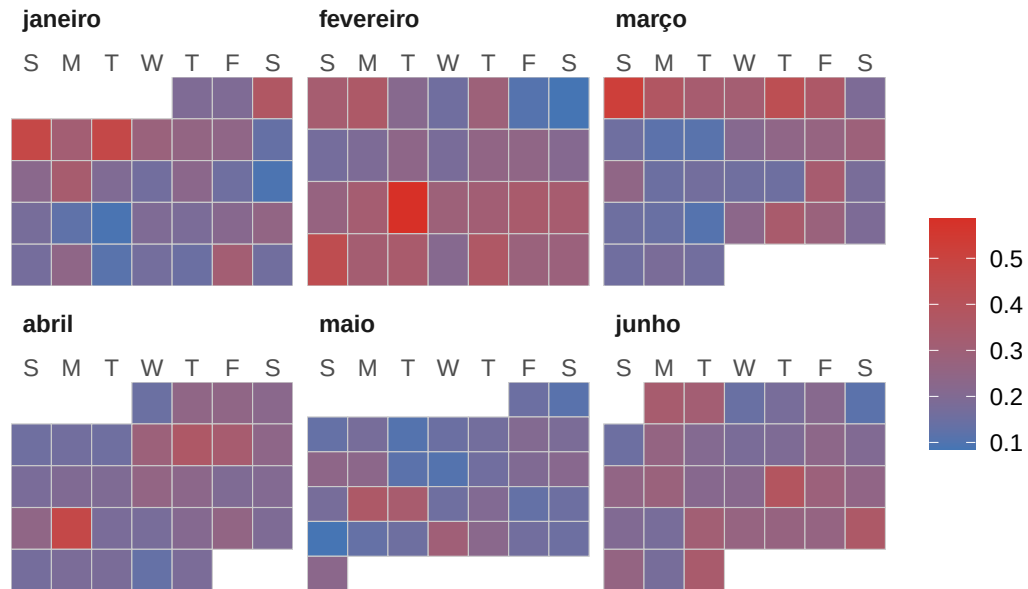
```
cUA + pal + ggtitle("Percentual de voos atrasados – UA")
```

Percentual de voos atrasados – UA



```
cUS + pal + ggtitle("Percentual de voos atrasados - US")
```

Percentual de voos atrasados – US



```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-08-24 23:45:57 -03"
```